وثيقة المراجعة والتلخيص الأكاديمي

ASE · Chapter 5 · Software Design

المستوى: Understand + Remember · **المصدر:** أسئلة الفصل الرسمية (Singh & Aggarwal) + إعادة صياغة بنفس النمط. **القاعدة:** كل جواب مأخوذ من النص (سلايد/كتاب) — بلا اختراع. الأرقام بالعمود تطابق ترقيم الكتاب.

الجزء أ — أسئلة الاختيار من متعدد (5.1 – 5.12)

5.1 The most desirable form of coupling is: (a) Control Coupling (b) Data Coupling (c) Common Coupling (d) Content Coupling **Answer:** (b) Data Coupling Data coupling is the best form; modules communicate only by passing primitive data items through parameters, with nothing else shared.

5.2 The worst type of coupling is: (a) Content coupling (b) Common coupling (c) External coupling (d) Data coupling **Answer:** (a) Content coupling Content coupling is the worst form, where one module modifies the data of another module or branches into the middle of it.

5.3 The most desirable form of cohesion is: (a) Logical cohesion (b) Procedural cohesion (c) Functional cohesion (d) Temporal cohesion **Answer:** (c) Functional cohesion Functional cohesion is the best form; every element of the module works together to perform a single, well-defined task.

5.4 The worst type of cohesion is: (a) Temporal cohesion (b) Coincidental cohesion (c) Logical cohesion (d) Sequential cohesion **Answer:** (b) Coincidental cohesion Coincidental cohesion is the worst; the module contains elements with little or no relationship to one another.

5.5 Which one is not a strategy for design? (a) Bottom up design (b) Top down design (c) Embedded design (d) Hybrid design **Answer:** (c) Embedded design The three design strategies are bottom-up, top-down, and hybrid. "Embedded design" is not one of them.

5.6 Temporal cohesion means: (a) Cohesion between temporary variables (b) Cohesion between local variable (c) Cohesion with respect to time (d) Coincidental cohesion **Answer:** (c) Cohesion with respect to time Temporal cohesion groups tasks simply because they must all be executed within the same time span (e.g. all initialisation at startup).

5.7 Functional cohesion means: (a) Operations are part of single functional task and are placed in same procedures (b) Operations are part of single functional task and are placed in multiple procedures (c) Operations are part of multiple tasks (d) None of the above **Answer:** (a) Operations are part of a single functional task and are placed in the same procedure.

5.8 When two modules refer to the same global data area, they are related as: (a) External coupled (b) Data coupled (c) Content coupled (d) Common coupled **Answer:** (d) Common coupled Common coupling occurs when two modules share the same global data area.

5.9 The module in which instructions are related through flow of control is: (a) Temporal cohesion (b) Logical cohesion (c) Procedural cohesion (d) Functional cohesion **Answer:** (c) Procedural cohesion Procedural cohesion

occurs when the instructions must be executed in a specific order of flow of control.

5.10 The relationship of data elements in a module is called: (a) Coupling (b) Cohesion (c) Modularity (d) None of the above **Answer:** (b) Cohesion Cohesion measures how strongly the elements *within* a module are related.

5.11 A system that does not interact with the external environment is called: (a) Closed system (b) Logical system (c) Open system (d) Hierarchal system **Answer:** (a) Closed system

5.12 The extent to which different modules are dependent upon each other is called: (a) Coupling (b) Cohesion (c) Modularity (d) Stability **Answer:** (a) Coupling

---

الجزء ب — الأسئلة المقالية الرسمية (Exercise 5.1 – 5.24)

5.1 What is design? Describe the difference between conceptual design and technical design. Answer: Design is the process of applying various techniques and principles to produce a model of a system before it is built. **Conceptual design** answers *what* the system will do — where the data comes from, what happens to data in the system, how the system looks to users, what choices are offered, event timing, and how reports and screens look. **Technical design** describes *how* the system will be realised — hardware configuration, software needs, communication interfaces, I/O, software architecture, and network architecture. Conceptual design is the "outside" view (requirements-oriented); technical design is the "inside" view (solution-oriented).

5.2 Discuss the objectives of software design. How do we transform an informal design to a detailed design? Answer: Objectives: the design must be **correct and complete, understandable, at the right level of abstraction, and maintainable**. Transformation: design passes through progressive refinement — an **informal design outline** → an **informal design** → a **more formal design** → the **finished design** (Fig. 3). Each stage adds detail while preserving the validity of the previous stage.

5.3 Do we design software when we "write" a program? What makes software design different from coding? Answer: Yes — even a small program involves design decisions, but coding is only the *translation* of the design into a programming language. Software design differs from coding because it deals with structure, module boundaries, interfaces and abstraction, and is independent of any programming language; coding is the implementation of an already-designed structure.

5.4 What is modularity? List the important properties of a modular system. Answer: Modularity is the single attribute of software that makes a program intellectually manageable. It consists of well-defined, manageable units with well-defined interfaces. Properties: (i) well-defined subsystem; (ii) well-defined purpose; (iii) can be separately compiled and stored in a library; (iv) a module can use other modules; (v) a module should be easier to use than to build; (vi) simpler from outside than from the inside.

5.5 Define module coupling and explain different types of coupling. Answer: Coupling is the measure of the degree of interdependence between modules. Best → worst: **Data** (only primitive data passed) → **Stamp** (whole data structure passed) → **Control** (control flag passed) → **External** (both modules tied to external format/protocol) → **Common** (shared global data area) → **Content** (one module reaches into another's internals).

5.6 Define module cohesion and explain different types of cohesion. Answer: Cohesion is a measure of the degree to which the elements of a module are functionally related. Best → worst: **Functional** (single task) →

Sequential (output of one is input of next) → **Communicational** (operate on the same data) → **Procedural** (related by order of execution) → **Temporal** (related by time) → **Logical** (same logical class) → **Coincidental** (no relationship).

5.7 Discuss the objectives of modular software design. What are the effects of module coupling and cohesion? Answer: Objectives: produce modules that are independently understandable, testable and maintainable, so the system is intellectually manageable. Effects: **low coupling** localises the ripple effect — a change in one module does not propagate; **high cohesion** makes each module focused, hence easier to understand and reuse. Together they give **functional independence**.

5.8 If a module has logical cohesion, what kind of coupling is this module likely to have with others? Answer: Logical cohesion typically brings **control coupling** and often **common coupling** — because the module needs a flag/parameter to decide which logical function to execute, and such "utility" modules frequently share global data.

5.9 What problems are likely to arise if two modules have high coupling? Answer: A change in one module forces changes in the other (**ripple effect**); errors propagate across the boundary; the modules cannot be understood, tested or reused independently; maintainability and reliability both drop sharply.

5.10 What problems are likely to arise if a module has low cohesion? Answer: The module has multiple unrelated responsibilities, so it is hard to name, understand, test and reuse; it changes for many different reasons; it usually drags in extra coupling and makes the whole design unstable.

5.11 Describe the various strategies of design. Which design strategy is most popular and practical? Answer: Bottom-up (build low-level modules first, collect into a library), **Top-down** (start from major modules, decompose by stepwise refinement), and **Hybrid** (top-down overall with bottom-up for common sub-modules, library reuse, and low-level modules). **Hybrid** is the most popular and practical.

5.12 If some existing modules are to be re-used in building a new system, which design strategy is used and why? Answer: Bottom-up, because it begins from available/pre-written modules and assembles them upward — that is exactly how a reusable library is exploited.

5.13 What is the difference between a flow chart and a structure chart? Answer: A **flow chart** shows the *logic/sequence of control* within a program (decision, loop, process). A **structure chart** shows the *hierarchical module decomposition* of a system — which module calls which, and what data/control passes between them. One is intra-module logic; the other is inter-module architecture.

5.14 Explain why it is important to use different notations to describe software designs. Answer: No single notation expresses every view of a design. Different notations (structure chart, DFD, transaction diagram, pseudocode/PDL, natural language) communicate different aspects to different readers (designer, developer, tester), support different design views, and improve clarity and correctness of the design description.

5.15 List a few well-established function oriented software design techniques. Answer: Structure charts, Data Flow Diagrams (DFD), transaction diagrams/transaction-centred structure, functional procedure layers, and pseudocode/PDL.

5.16 Define the following terms: Objects, Message, Abstraction, Class, Inheritance and Polymorphism. Answer: Object — an entity that retains a state and offers operations to examine/affect that state. **Message** — a request sent from one object to another, consisting of the target object identity, the operation name, and any

needed parameters. **Abstraction** — eliminating the irrelevant and amplifying the essentials. **Class** — a set of objects sharing a common structure (attributes) and behaviour (operations). **Inheritance** — a subclass acquires state and behaviour from a superclass. **Polymorphism** — abstracting just the interface of an operation and leaving the implementation to subclasses.

5.17 What is the relationship between abstract data types and classes? Answer: An ADT is a *specification* of data plus the operations on it, with implementation hidden. A class is the *implementation* realisation of that idea — it binds data (attributes) and operations (methods) together and can instantiate objects. In short: a class is a concrete, instantiable ADT.

5.18 Can we have inheritance without polymorphism? Explain. Answer: Yes. Inheritance is the mechanism of a subclass acquiring structure/behaviour from a superclass. Polymorphism is the ability to treat subclasses through a common interface with different implementations. A class can inherit without any operation being overridden polymorphically — so inheritance can exist without polymorphism.

5.19 Discuss the reasons for improvement using object-oriented design. Answer: OOD models the problem in terms of real-world objects, so the model is more natural and stable against requirement changes; encapsulation localises change; inheritance and polymorphism promote reuse and extensibility; the resulting system is easier to maintain, extend and understand, and the design is independent of any implementation language.

5.20 Explain the design guidelines that can be used to produce "good quality" classes or reusable classes. Answer: Keep classes focused on a single responsibility; hide implementation details (encapsulation); minimise coupling between classes and maximise cohesion inside them; give classes a clear, meaningful name and a small, complete interface; design for reuse (general, parameterised behaviour); avoid deep inheritance chains; and keep attributes private with controlled access.

5.21 List the points of a simplified design process. Answer: (i) Create the use case model; (ii) draw the activity diagram (if required); (iii) draw the interaction diagrams; (iv) draw the class diagram; (v) draw the state chart / object diagram (if required); (vi) draw the component and deployment diagrams — after which the design documents are complete. *(Six points — as printed in the book's Fig 5.25 and reproduced on slide 72.)*

5.22 Discuss the differences between object oriented and function oriented design. Answer: Function-oriented design decomposes the system by **function** — what the system does — producing a hierarchy of modules. Object-oriented design decomposes by **data/objects** — the things the system manipulates — producing interacting objects. They are orthogonal: one focuses on the function performed, the other on the data manipulated. OOD is generally more natural for modelling real-world problems.

5.23 What documents should be produced on completion of the design phase? Answer: The **Software Design Description (SDD)** — per IEEE STD 1016 — containing the design views: decomposition description, dependency description, interface description, and detail description, plus the design entity attributes (identification, type, purpose, function, subordinates, dependencies, interface, resources, processing, data).

5.24 Can a system ever be completely "decoupled"? That is, can the degree of coupling be reduced so much that there is no coupling between modules? Answer: No. Any two modules that collaborate must exchange *something* — at minimum data through a call/interface — so some coupling always remains. The realistic goal is to reduce coupling to its **best form (data coupling)**, not to eliminate it; zero coupling would mean the modules never interact, which is not a working system.

المصدر: Singh & Aggarwal, Software Engineering 3rd ed., Ch.5 — سلايدات 1–99 + كتاب (ص.202–251). كل جواب مُتحقق منه بالنص.